TajwayApp

Project Documentation — UI, Functionality & API Integration

Product: Tajway — India's B2B Taxi Marketplace

Tagline: Bharat Ki Apni Taxi Community

Technology: React Native, React Navigation, Redux Toolkit, Axios

API Base URL: `https://cytopathogenic-harmony-prosurrender.ngrok-free.dev/api/v1/app`

Document Date: October 2026

Tajway is a B2B mobile application for India's taxi and travel industry. It connects three roles on one platform: travel agents who need cars for customers, fleet owners who supply vehicles and drivers, and drivers who take trips. In simple words, it is a marketplace where booking demand and vehicle supply can meet.

Today, the Agent side is the most complete. An agent can log in with OTP, browse the market, post a booking or a free vehicle, open chat screens, and use profile and verification UI. Driver and Owner sides mainly have starting screens and are not full products yet.

Contents

1. UI and Functionality
2. API Integration
3. Summary Checklist

1. UI and Functionality

1.1 Overall Progress

The project has many screens, but not every screen is finished to the same level. Some screens are complete and connected to real APIs. Some look complete but still use mock data. Some are only basic placeholders.

Area	What it means	Approx. completion
Agent journey (Market, Post, Profile)	Main product path. Most screens are designed and usable for demos.	70–75% UI
Live data behind Agent screens	Only selected actions talk to the server. Many lists still use sample data.	35–40% API-backed
Driver / Owner journey	After OTP, users see a simple welcome/home screen with logout.	5–10%

1.2 Application Flow

When the app opens, the user first sees branding, then a short introduction, then phone login. After OTP success, the app checks the user's role from the server response and opens the matching home.

```
Splash → Intro slides → Phone login → OTP verification
    → Agent home (full tab application), or
    → Driver home (scaffold), or
    → Owner home (scaffold)
```

Agent bottom tabs:

Tab	Purpose
Market	Browse public bookings and free vehicles.
My Bookings	See bookings the agent posted or received.
+ (center)	Create a new booking or free vehicle listing.
Chat	Open the message inbox related to bookings.
Profile	Manage account, verification, network, fleet links, and logout.

From these tabs, the user can open extra screens such as booking detail, personal information, Aadhaar verification, KYC verification, route alert setup, and chat conversation.

1.3 Onboarding and Authentication

Authentication is phone-based. The live flow is simple: enter mobile number, receive OTP, verify OTP, and enter the app.

Screen	Purpose	Backend
SplashScreen	Brand loading and startup routing	Local only
OnboardingScreen	Product intro before login	Local only
PhoneLoginScreen	Collect 10-digit mobile number and request OTP	API — send OTP
OTPScreen	Verify OTP, save session, open role home	API — verify OTP

SplashScreen

- This is the first screen after the app starts. It shows the Tajway logo and brand message.
- It displays branding and a short animation for about two seconds.
- After that delay, it moves the user to the intro / onboarding slides.
- Its purpose is branding and startup routing, not data entry.

OnboardingScreen

- Introduces the product before login so first-time users understand what Tajway is.
- Shows a carousel of intro slides with artwork and short product messages.

- Slides can auto-advance so the user can watch without tapping every time.
- When the user taps Get Started, the app opens Phone Login.

PhoneLoginScreen

- Login begins here. The user enters a 10-digit Indian mobile number.
- The field accepts only digits and is limited to 10 characters.
- If fewer than 10 digits are entered, the app shows a notice asking for a valid number.
- On Continue, the app calls `sendOtp` with the mobile number.
- If the OTP request succeeds, the phone number is saved in Redux and the OTP screen opens.
- If the request fails, an error notice is shown and the user stays on this screen.

OTPScreen

- This screen completes login. The user enters the 6-digit OTP received on the phone.
- It shows a masked phone number so the user can confirm where the OTP was sent.
- OTP is entered in a 6-box input. When all digits are filled, verification can run.
- The screen calls `verifyOtp` with mobile and OTP.
- On success, the server returns user details, role, and tokens. These are saved in AsyncStorage (`auth_session`) and also stored in Redux.
- Based on role (`agent`, `driver`, or `owner`), navigation resets to the matching root screen.
- A resend timer allows requesting another OTP after a wait period.

1.4 Marketplace (Market Tab)

Market is the heart of the Agent app. Agents come here to discover available bookings and free vehicles posted by other partners. This screen also nudges incomplete profiles toward verification and setup.

Screen	What the user can do	Data status
MarketHomeScreen	Browse bookings / free vehicles, search, filter, setup progress, tour, help	Bookings live if verified; otherwise mock. Free vehicles list is mock.
MarketBookingDetailScreen	View full trip summary, call partner, open chat	UI complete; no booking-by-id API
RouteAlertSetupScreen	Set preferred vehicle and routes for alerts	UI complete; save is local only

MarketHomeScreen — detailed behavior

- **Tabs:** The user can switch between Bookings and Free Vehicles. Bookings are trip requests. Free Vehicles are cars available for hire in a time window.
- **Search and filters:** The user can search text and open a filter sheet for trip type, vehicle type, and pickup/drop preferences. Filters narrow the visible list on the device.
- **Verification-based data:** When Market opens, it fetches the latest user profile with `UserService.getMe()`. If verification is complete, bookings are loaded from `GET /bookings`. If the user is not verified, the screen shows sample market bookings from mock data.

- **Free Vehicles tab:** Creating a free vehicle can go to the API from the Post flow, but the Free Vehicles list on Market still reads from mock data. So new posts may not appear here from the server yet.
- **Setup progress:** A progress ring / checklist guides the user through personal info, verification, and related setup steps.
- **Verification prompt:** If the user is not verified, a bottom sheet may appear after a short delay asking them to complete KYC.
- **App tour:** First-time users can get a spotlight tour that highlights important Market controls, with spoken guidance. Completion is saved on the device.
- **Help / call:** A help action can open the phone dialer with a sample support number.

MarketBookingDetailScreen

- Opened when a user taps a booking card on Market or My Bookings.
- Shows route, dates, vehicle type, amount/quote details, and partner information.
- Call opens the device dialer. Chat navigates into the chat conversation UI.
- Booking content is passed through navigation parameters. There is no “get booking by id” API yet.

RouteAlertSetupScreen

- Helps an agent set which routes and vehicle types they care about for future matching.
- It is a step-by-step setup for vehicle choice, one-way routes, and round-trip routes.
- Locations can be selected through Location Select.
- Saving currently keeps values on the device / local UI state. There is no route-alert API yet.

1.5 Bookings and Posting

Agents both consume and create supply/demand. My Bookings is for managing existing posts. The center “+” tab is for creating new listings.

Screen	Purpose	Backend
MyBookingsScreen	Private desk for posted / received bookings, accept/reject	Mock / local
PostBookingScreen	Publish a new trip request to the marketplace	API — POST /bookings
PostFreeVehicleScreen	Advertise an available vehicle for a time window	API — POST /bookings
LocationSelectScreen	Pick city / airport for forms	Static local list

MyBookingsScreen

- This is the agent’s private booking desk, separate from the public Market feed.
- It has Posted and Received tabs so the agent can see bookings they created and bookings coming to them.
- Search and filters work similarly to Market for narrowing the list.
- Each booking card can open detail, share, or show accept/reject actions where relevant.
- Accept and reject open a confirmation modal. After confirm, status is updated in local state only.

- The list itself is loaded from mock data, not from a “my bookings” API.

PostBookingScreen

- Lets an agent publish a new trip request. This create flow already talks to the backend.
- The agent chooses one-way or round-trip.
- They select pickup and drop locations, date and time, vehicle type, and pricing style (fixed amount or quote).
- Extra options and visibility settings can also be filled.
- On submit, the app builds a payload and calls `BookingService.createBooking`, which posts to `/bookings` with `listingType: 'new_booking'`.
- Success and failure are handled with UI feedback so the user knows whether the post was created.

PostFreeVehicleScreen

- Used when the agent has a free car and wants to advertise availability.
- The user selects vehicle type, available time window, location, and optional details.
- On submit, the app calls `BookingService.createFreeVehicle`, which posts to `/bookings` with `listingType: 'free_vehicle'`.
- Even though create works on the API, the Market Free Vehicles tab still shows mock list data, so end-to-end list refresh is incomplete.

LocationSelectScreen

- Many booking and alert forms need a pickup or drop location. This screen provides a simple picker.
- The user can search among popular cities and airports from a static in-app list.
- Selecting a location returns the value to the previous form and goes back.
- There is no Google Places / Maps autocomplete API yet.

1.6 Chat and Profile

Chat and Profile support the booking lifecycle after discovery. Chat is for partner communication. Profile is for account trust, network, and settings.

Screen	Purpose	Backend
AgentMessagesScreen	Chat inbox for posted / received conversations	Mock
ChatDetailsScreen	One conversation thread with booking summary	Mock / local send
AgentProfileScreen	Account hub and menu of profile-related screens	Uses Redux user
PersonalInfoScreen	Edit name, city, company, email, role	API — PATCH profile
AadhaarVerifyScreen	Aadhaar number + OTP style confirmation UI	Local validation only
VerificationScreen	Full multi-step KYC wizard	UI only (submit not API)
ManageVehicles / ManageDrivers	Fleet empty states and add forms	UI shell

Screen	Purpose	Backend
MyNetworkScreen	Partner contact directory	Mock / local add
PaymentMethodsScreen	Bank account form for payouts	UI only
TransactionsScreen	Payment history list	Mock
Verified Supplier screens	Plan / benefits marketing pages	UI stubs

Chat screens

- **AgentMessagesScreen:** Shows chat rows with search and filters. Tapping a row opens Chat Details. All rows currently come from mock data.
- **ChatDetailsScreen:** Shows the message thread, a short booking summary, and a compose box. Sending a message updates local UI only; messages are not synced to a server.

Profile and account screens

- **AgentProfileScreen:** Shows banner, avatar, and a menu for personal information, network, drivers, vehicles, payments, transactions, verified supplier, verification, and logout. Some items show Coming Soon.
- **PersonalInfoScreen:** Editable fields include full name, city, company name, email, and role. Save calls `UserService.updateProfile` (PATCH `/user/profile`). Photo/GST/Aadhaar are not fully backend-driven yet.
- **AadhaarVerifyScreen:** Collects a 12-digit Aadhaar number and a short OTP. Checks are local format/UI validation only.
- **VerificationScreen:** Walks through profile, GST/Aadhaar, bank details, and vehicle/driver documents. Final submit currently shows an alert. There is no verification-submit API. Market still uses verification status from profile (`getMe`) to decide whether to show live bookings.
- **ManageVehicles / ManageDrivers:** Empty states with add forms. Values are not saved through an API yet.
- **MyNetworkScreen:** Contact cards with invite/status style information and an add-contact modal. Initial contacts are mock; new contacts stay in local screen state.
- **PaymentMethodsScreen:** Bank account form. Update currently navigates back without a payment API save.
- **TransactionsScreen:** Polished transaction cards loaded from mock data.
- **Verified Supplier screens:** Explain plan pricing and benefits. Purchase/upgrade actions are stubs, not a payment gateway.

1.7 Driver and Owner

The app already routes Driver and Owner roles after OTP, but their product features are not built out yet. These homes mainly prove that role-based navigation works.

Screen	Current behavior	Status
DriverHomeScreen	Welcome card, empty jobs area, sign out	Scaffold
OwnerHomeScreen	Welcome message, “coming soon”, sign out	Scaffold

- There is no live driver job feed, interest flow, or trip tracking on Driver Home yet.
- Owner Home does not yet include fleet dashboards, request management, or driver assignment.

1.8 How Core Features Work

This section explains the main product behaviors as end-to-end stories, so it is clear how screens and APIs connect.

OTP login

- User enters phone number on Phone Login.
- App calls send-OTP. Server sends an OTP to that number.
- User enters OTP on OTP screen. App calls verify-OTP.
- Server returns user profile, role, and tokens.
- App stores session locally, updates Redux, and opens Agent, Driver, or Owner home.

Verification gate on Market

- Market asks the server for the latest user profile.
- If the profile is marked verified, Market loads live bookings from the bookings API.
- If not verified, Market shows sample bookings and may prompt the user to finish verification.
- This protects the live marketplace while still letting new users explore the UI.

Create listing

- User taps the center “+” button on the Agent tab bar.
- A quick action opens either Post Booking or Post Free Vehicle.
- User fills the form and submits.
- App sends a POST request to `/bookings` with the correct listing type.
- On success, the listing is created on the server. Whether it immediately appears in Market depends on verification and list wiring (especially for free vehicles).

App tour, filters, accept/reject

- **App tour:** First Market visit can show a spotlight tour with spoken steps. Completion is stored locally.
- **Filters:** Market, My Bookings, and Chat share filter concepts for trip type, vehicle, and locations. Filtering happens on already loaded data.
- **Accept / reject:** On My Bookings, a confirmation modal updates local status only. A decision API is not connected yet.

1.9 Real Data vs Mock Data

Understanding this split is important for testing. If a screen uses mock data, server changes will not appear there, and actions may not persist after app restart.

Feature	Data source
OTP login	Real API
Market bookings (verified user)	Real API
Market bookings (unverified user)	Mock sample list
Free vehicles list on Market	Mock
Create booking / create free vehicle	Real API
My Bookings list and decisions	Mock / local
Chat inbox and conversation	Mock / local
Network and Transactions	Mock
Verification submit, Aadhaar, Payments, Vehicles, Drivers	UI only

1.10 State Management

- **authSlice:** Stores login status, user object, role, phone, tokens, and related flags. This is the most important shared state and is used after OTP, on profile screens, and on Market.
- **Other slices (booking, marketplace, driver, notification):** These exist for future or legacy flows. Current Agent Market and My Bookings screens mostly keep local state and call services directly.

2. API Integration

2.1 How the API Layer Works

Screens do not own all HTTP logic. They call service functions. Services call shared HTTP helpers. Helpers talk to the base URL.

```
Screen → Service (auth / user / booking)
       → Axios helpers in api/common.tsx
       → Base URL + path from endpoints.tsx
```

- **Axios setup:** One shared Axios instance uses the project base URL, a 30-second timeout, and JSON headers. An ngrok skip-browser-warning header is included so free ngrok tunnels return JSON instead of an HTML warning page.
- **Auth header:** For protected routes, helpers read `auth_session` from AsyncStorage and attach `Authorization: Bearer <token>`.
- **Response shape:** Services generally expect `{ success, message?, data? }`. If `success` is false, they treat it as an error and surface a message to the UI.

- **Still missing:** Automatic token refresh, global 401 interceptors, and a DELETE helper are not implemented yet.

2.2 Defined Endpoints

All known backend paths used by the app are listed in `src/api/endpoints.tsx`. Some are already connected to screens. Some are only reserved for later.

Endpoint key	Path	Used in app?	Where
SEND_OTP	`/auth/send-otp`	Yes	PhoneLoginScreen
VERIFY_OTP	`/auth/verify-otp`	Yes	OTPScreen
USER_ME	`/user/me`	Yes (GET)	MarketHomeScreen
UPDATE_PROFILE	`/user/profile`	Yes (PATCH)	PersonalInfoScreen
BOOKINGS	`/bookings`	Yes (GET + POST)	Market list, Post Booking, Post Free Vehicle
LOGIN	`/auth/login`	No	Reserved (OTP is used instead)
LOGOUT	`/auth/logout`	No	Reserved (logout is mostly local)
REFRESH_TOKEN	`/auth/refresh-token`	No	Reserved (token stored, not refreshed)

2.3 Services

Services are the business layer between UI and HTTP. They validate inputs, call endpoints, map responses, and update local session storage where needed.

Auth service

- `sendOtp(mobile)` validates that the number looks like a 10-digit Indian mobile, then POSTs `{ mobile }` to send-OTP.
- `verifyOtp({ mobile, otp })` POSTs mobile and OTP to verify-OTP. On success it stores access token, refresh token, user, role, and related flags in `auth_session`.
- `loadSession` reads the saved session from storage. `clearSession` removes it.

User service

- `getMe()` GETs the current user from `/user/me` and merges the latest user object into the stored session.
- `updateProfile(...)` PATCHes selected profile fields to `/user/profile` (full name, role, city, company name, optional email).
- Helper `isVerificationComplete(user)` checks client-side flags such as verified badge / verification status so Market can decide whether to load live listings.

Booking service

- `getMarketBookings(...)` GETs `/bookings` with query params such as listing type, page, and limit, then maps each item into the card format expected by Market UI.
- `createBooking(form)` converts the Post Booking form into an API payload and POSTs it with `listingType: 'new_booking'`.
- `createFreeVehicle(form)` does the same for free vehicles with `listingType: 'free_vehicle'`.

Driver service

- Provides mock helpers for available bookings, showing interest, and trip status changes.
- It uses local storage and sample bookings, not live driver APIs.
- No active Driver home screen currently imports this service.

2.4 Feature Integration Status

Feature	Status	Notes
OTP authentication	Integrated	Primary and only live login path
Profile	Partial	GET me + PATCH profile; photo/GST/Aadhaar not fully API-driven
Create booking / free vehicle	Integrated	POST /bookings
Market booking list	Partial	Live for verified users; mock for others
Free vehicle list	Mock	Create API exists; list feed not wired
My Bookings	Mock	Local accept/reject
Chat / Network / Payments / Vehicles	Mock / UI	No backend calls
Verification submit	UI only	Wizard complete; no submit API
Driver jobs	Mock	Service unused by active screens
Logout / refresh token	Not wired	Endpoint constants exist only

2.5 Screens Connected to the Backend

Only a small set of screens currently make real HTTP calls. Everything else is either mock-driven or local UI.

Screen	API used
PhoneLoginScreen	POST /auth/send-otp
OTPScreen	POST /auth/verify-otp (including resend)
MarketHomeScreen	GET /user/me; GET /bookings when verified
PersonalInfoScreen	PATCH /user/profile

Screen	API used
PostBookingScreen	POST /bookings (new booking)
PostFreeVehicleScreen	POST /bookings (free vehicle)

That means about six screens are backend-connected. Overall end-to-end API coverage for the full product vision is roughly 20–25%.

2.6 Auth Session Flow

Session handling has two layers: AsyncStorage for persistence and Redux for in-memory app state.

```
PhoneLogin → sendOtp
  → OTPScreen → verifyOtp
      → save auth_session (tokens + user)
      → Redux loginSuccess
      → open Agent / Driver / Owner home
```

- After verify-OTP, both storage and Redux are updated, so authenticated API calls and UI state work in the same session.
- On a full app restart, `loadSession` is available but is not consistently used to rehydrate Redux. Because of that, the UI may behave as logged out even if a token file still exists on disk.
- Logout is inconsistent across screens. Some clear only Redux. Some also clear `auth_session`. The server logout endpoint is not called.
- Refresh token is saved during login, but there is no automatic refresh call when access token expires.

2.7 API Completion Overview

Layer	Approx. completion	Comment
HTTP client setup	~75%	Shared Axios helpers and error handling; missing interceptors/refresh
Endpoint catalog	~40%	Five of eight defined paths are in active use
Service coverage	~35%	Auth, user profile, booking create/list
Screens on API	~12–15%	Six screens with real calls
Product end-to-end	~20–25%	OTP + profile + post/list bookings; most other flows still UI/mock

3. Summary Checklist

3.1 UI — Agent Path

- [x] Splash, onboarding, phone login, and OTP
- [x] Market home with tour, filters, and tabs
- [x] Post booking and free vehicle forms

- [x] My Bookings, Chat, and Profile hub
- [x] Personal info and verification wizard UI
- [ ] Full Driver and Owner applications
- [ ] Working fleet save for vehicles and drivers
- [ ] Live chat, payments, and transactions

3.2 API

- [x] send-otp / verify-otp
- [x] GET /user/me and PATCH /user/profile
- [x] GET /bookings for verified market users
- [x] POST /bookings for new booking and free vehicle
- [ ] My Bookings CRUD and decision APIs
- [ ] Free vehicles list feed on Market
- [ ] Chat, network, and KYC submit
- [ ] logout / refresh-token
- [ ] Driver and registration APIs

Prepared from the TajwayApp source tree (screens, API layer, services, navigation, mock data, and Redux).